

PaddleOCR With OpenCV for Unity 1.0.2



About PaddleOCR With OpenCV for Unity

PaddleOCR With OpenCV for Unity is a Unity plugin that combines **PaddleOCR** with **OpenCV for Unity** and **Unity Sentis** to deliver end-to-end **GPU** text **OCR**—**detection**, **classification**, and **recognition**—on multiple platforms with the same **C#** codebase.

PaddleOCR With OpenCV for Unity uses **PaddleOCR** models under the **Apache License 2.0**; see Third-Party Notices.txt file in package for details.

The execution of this asset requires **OpenCV for Unity 3.0.4** or later.

Requires Unity 6000.0.83f1 or later.

Requires **Unity Sentis** (com.unity.ai.inference).

Requires .NET Standard 2.1.

Inference uses **ONNX** models converted from official **PaddleOCR 3.0 (PP-OCRv5)** distributions and runs them through **Unity Sentis** (`.sentis` models) via **OpenCV for Unity's** `MultiBackendDnn` integration.

No network connection or cloud server access is required at runtime. All **OCR** inference runs **entirely on-device** using models bundled in `StreamingAssets`.

Our Asset Features

Cross Platform

- **iOS & Android** support.
- **Win & Mac & Linux** Standalone support.
- **WebGL** support.
- **ChromeOS** support.
- **visionOS** support. (beta)
- Support for preview in the **Editor**.

OCR Pipeline

- **Text Detection**: Detect text regions in images using **PP-OCRv5** mobile detection models with configurable letterbox input sizes (**320×320**, **640×640**, **1280×1280**).
- **Text Classification**: Classify text line orientation (0° / 180°) using **PP-LCNet** classification models.
- **Text Recognition**: Recognize text in cropped regions using **PP-OCRv5** mobile recognition models and character dictionaries.
- **Multi-language Recognition**: Supports multiple recognition language packs under `StreamingAssets` (e.g. **multi**, **en**, **latin**, **korean**, **cyrillic**, **arabic**, **devanagari**, **th**, **el**, **eslav**, **te**, **ta**).
- **Fully Local Inference**: No internet connection or cloud **API** calls are needed during normal operation. Detection, classification, and recognition all run locally on the target device.

Easy to Use

- `PaddleOCRComponent` : A single **MonoBehaviour** that loads models, runs the full **det** → **cls** → **rec** pipeline, and exposes **UnityEvent** / parsed results.
- `TextDetector` , `TextClassifier` , `TextRecognizer` : Lower-level classes for custom pipelines using **OpenCV's** `Mat` and `dnn` APIs.

- Input from `Texture2D`, image file paths, byte arrays, and **OpenCV's** `Mat` class (via `PaddleOCRComponent.Submit` overloads).
- Switch between **synchronous** and **asynchronous** inference via `PaddleOCRComponent.UseAsyncInference`.
- Switch inference between **Unity Sentis** (`.sentis` models) and **OpenCV's dnn module** (`.onnx` models) via `PaddleOCRComponent.InferenceBackend`.

Include Many Examples

- Includes a wide variety of example usage scenarios, which consist of **scene files** and **script codes**. By running these example applications, **you can learn how to develop PaddleOCR applications effectively**.
- **TutorialExample**: Minimal tutorial using `PaddleOCRComponent.Submit(Texture2D)` and parsed results.
- **BenchmarkExample**: OCR benchmark on an assigned `Texture2D` with timing and visualization.
- **MultiSourceToMatHelperExample**: Real-time OCR from multiple video sources via `MultiSourceToMatHelper`.
- **HandwritingOCRExample**: Handwriting canvas with periodic and on-demand OCR.

Quick Links

- **Official Site** - Main product page
- **Example Code** - GitHub repository with examples
- **Android Demo** - Downloadable Android demo
- **WebGL Demo** - Online WebGL demo

System Requirements

Platform	Requirement
Windows Standalone & Editor	Windows 8 or later
Mac Standalone & Editor	macOS 10.13 or later
Linux Standalone & Editor	Ubuntu 18.04 or later

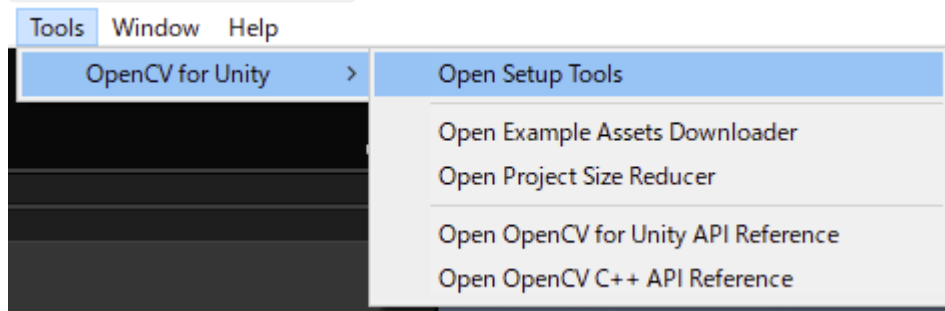
Platform	Requirement
Android	API level 24 or later
iOS	iOS Version 12.0 or later
visionOS (beta)	visionOS 1 or later

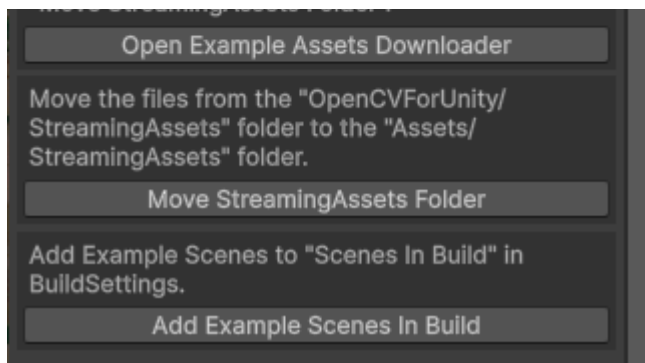
Setup Guide for Getting Started

1. Import **OpenCV for Unity** from the **Package Manager** and complete the [OpenCV for Unity Setup Guide](#).

⚠ Important: If your project already has a previous version of the Assets/OpenCVForUnity folder, issues may occur. Please delete the Assets/OpenCVForUnity folder first before importing the new version.

2. Select the menu item **Tools > OpenCV for Unity > Open Setup Tools** to open the Setup Tools window. Click the **Enable Unity Sentis Integration** button to install **com.unity.ai.inference** (**required** for this asset).

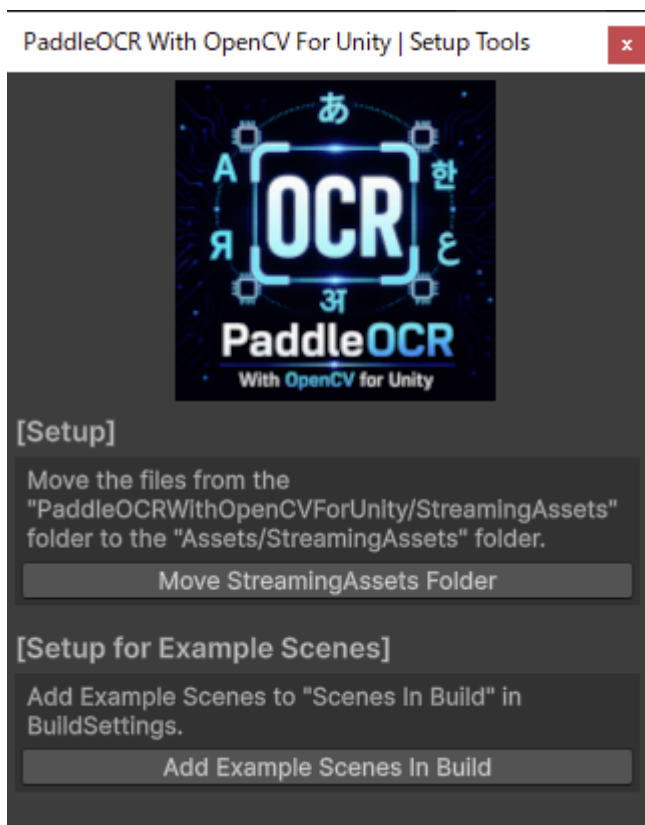
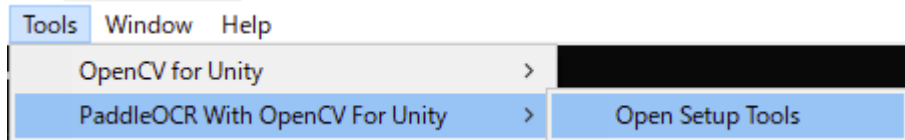




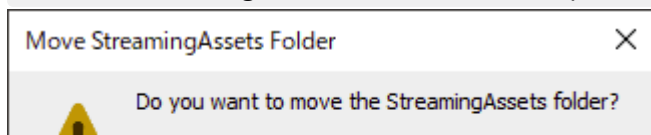
3. Import **PaddleOCR With OpenCV for Unity** from the **Package Manager** .

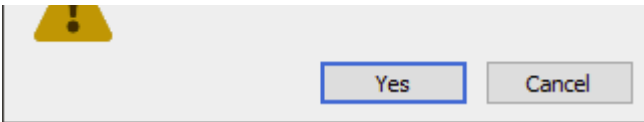
⚠ Important: If your project already has a previous version of the `Assets/PaddleOCRWithOpenCVForUnity` folder, please delete it first before importing the new version.

4. Select the menu item **Tools > PaddleOCR With OpenCV For Unity > Open Setup Tools** to open the **Setup Tools** window.



5. Click the **Move StreamingAssets Folder** button in the **Setup Tools** window to move the `Assets/PaddleOCRWithOpenCVForUnity/StreamingAssets` folder contents to `Assets/StreamingAssets` (for example, model files under `Assets/StreamingAssets/PaddleOCRWithOpenCVForUnity/`).

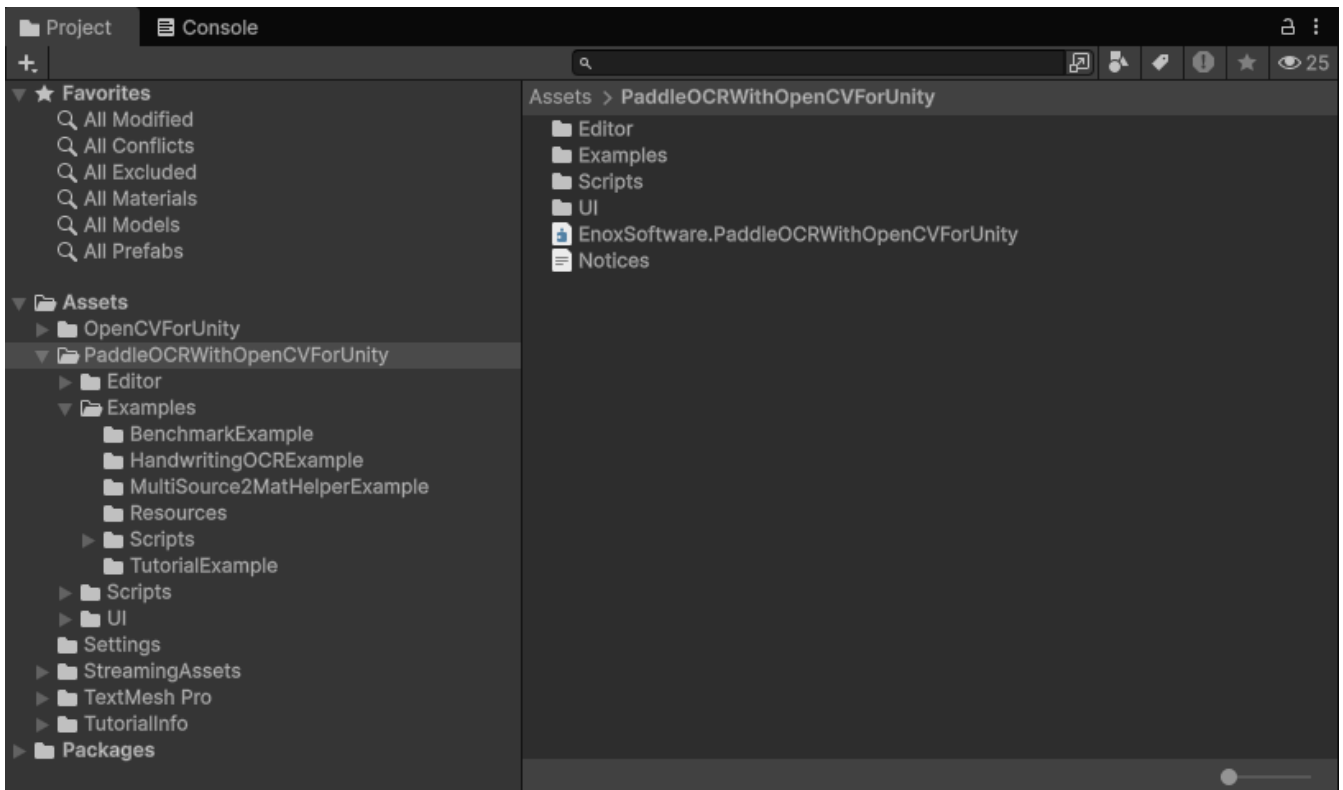




If you add or replace `.sentis` / **ONNX** models later, place them under `Assets/StreamingAssets/PaddleOCRWithOpenCVForUnity/` (see [PaddleOCR Model Files](#) and [PaddleOCR Model Conversion](#)).

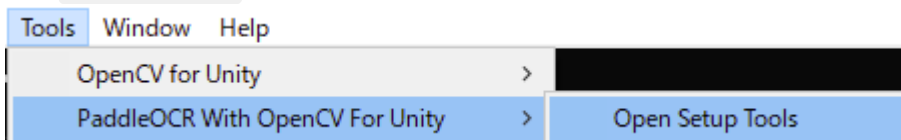
6. Setup is now complete.

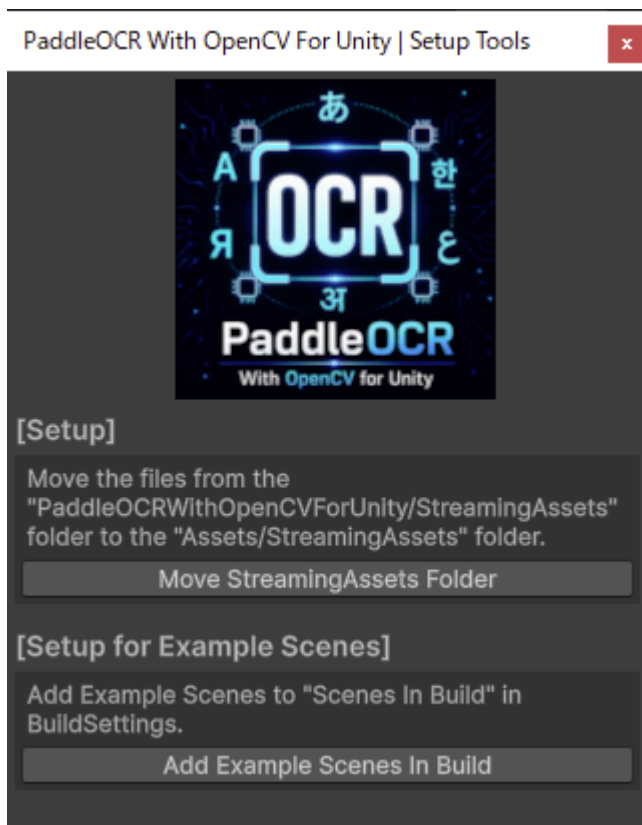
As a starting point for development using this asset, we recommend first running **TutorialExample** and then exploring the other Example Scenes.



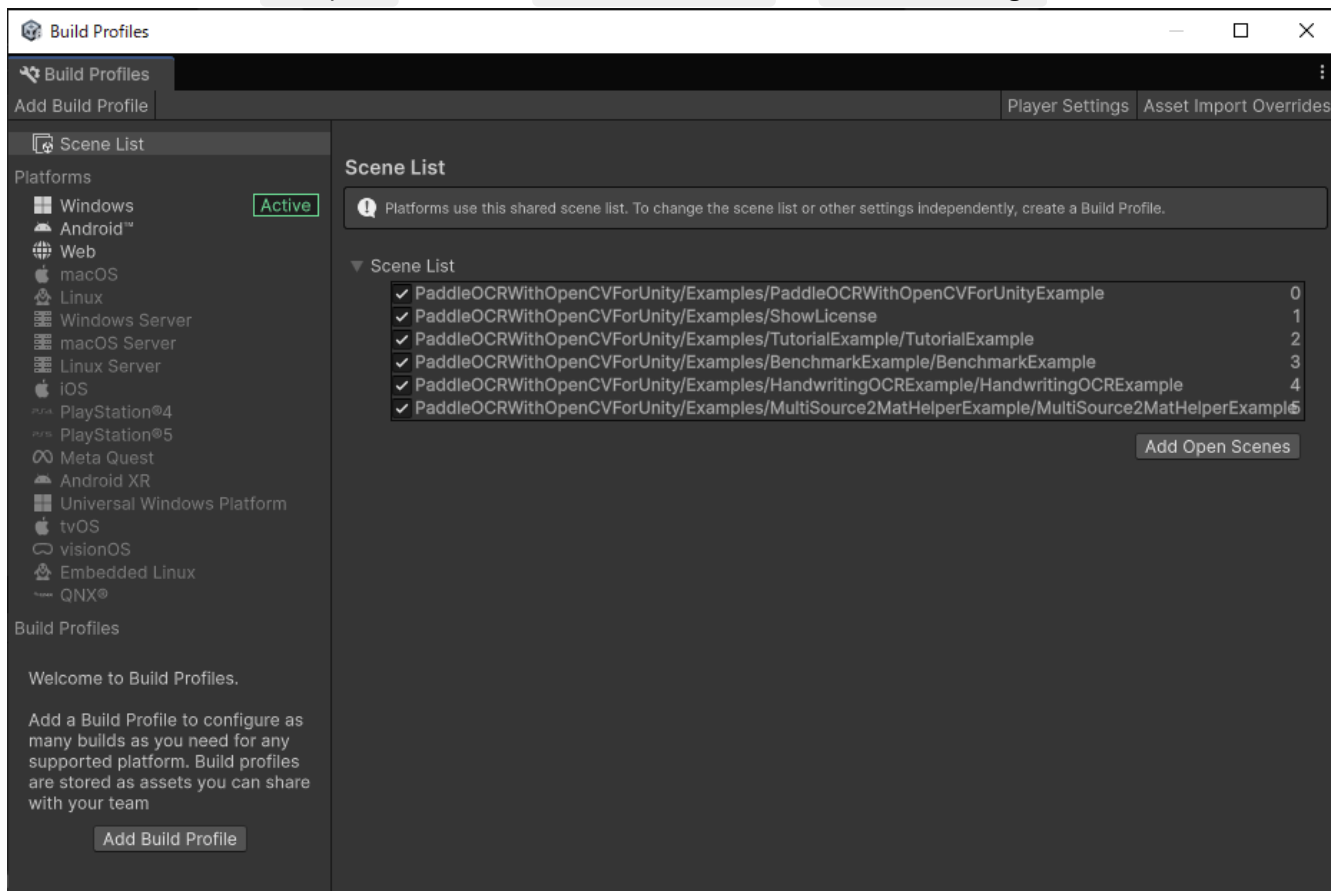
Example Scenes Setup Guide

1. Complete the [Setup Guide for Getting Started](#).
2. Confirm **PaddleOCR** model files exist under `Assets/StreamingAssets/PaddleOCRWithOpenCVForUnity/` (default root folder name used by `PaddleOCRComponent`). Complete step 5 of the [Setup Guide for Getting Started](#) if you have not moved the `StreamingAssets` folder yet.
3. Select the menu item `Tools > PaddleOCR With OpenCV For Unity > Open Setup Tools` to open the `Setup Tools` window.





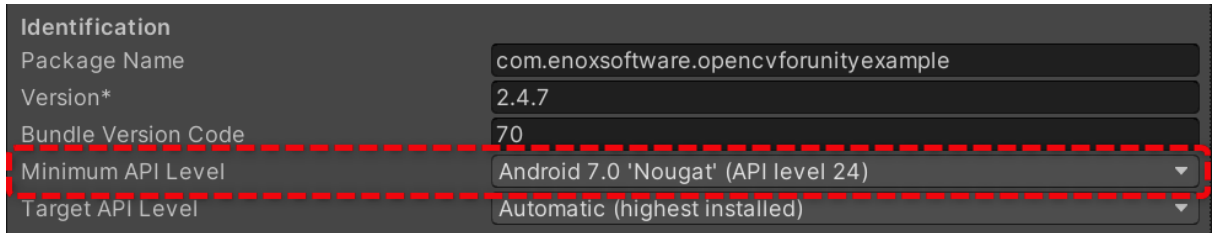
4. Click the **Add Example Scenes In Build** button in the **Setup Tools** window to add all Example Scene files in the **Examples** folder to **Scenes In Build** in **Build Settings**.



5. When building Example Scenes and running them on devices, settings may need to be changed depending on the platform.

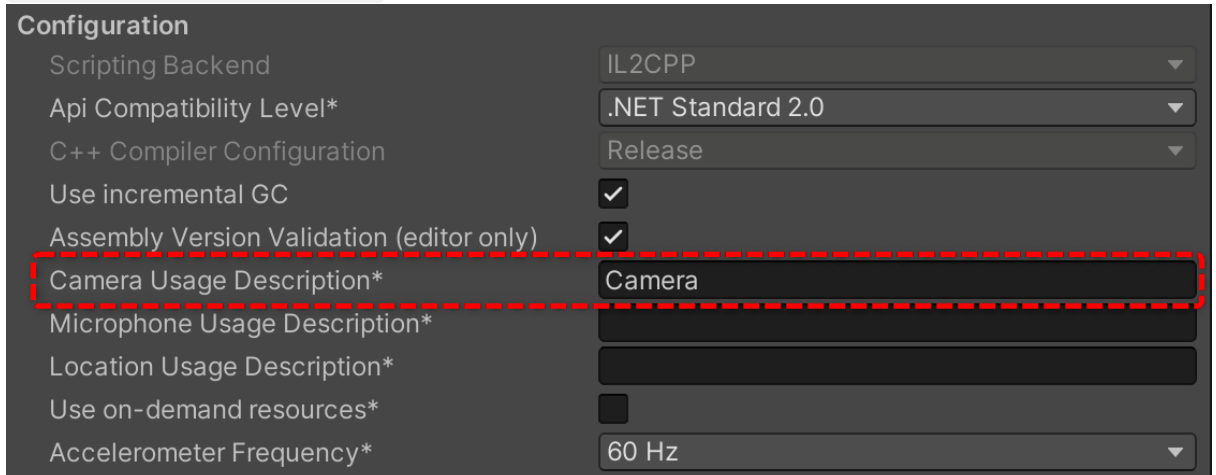
- **Android**

- Set **Player Settings** → **Other Settings** → **Identification** → **Minimum API Level** to **24** or higher.



• iOS

- Set **Player Settings** → **Other Settings** → **Configuration** → **Camera Usage Description** when using camera input.

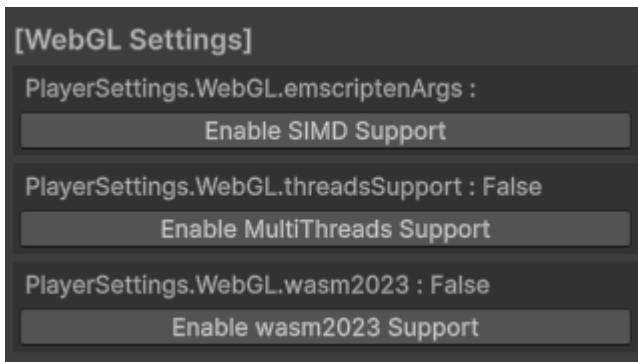


- Set **Target minimum iOS Version** to **12.0** or higher.

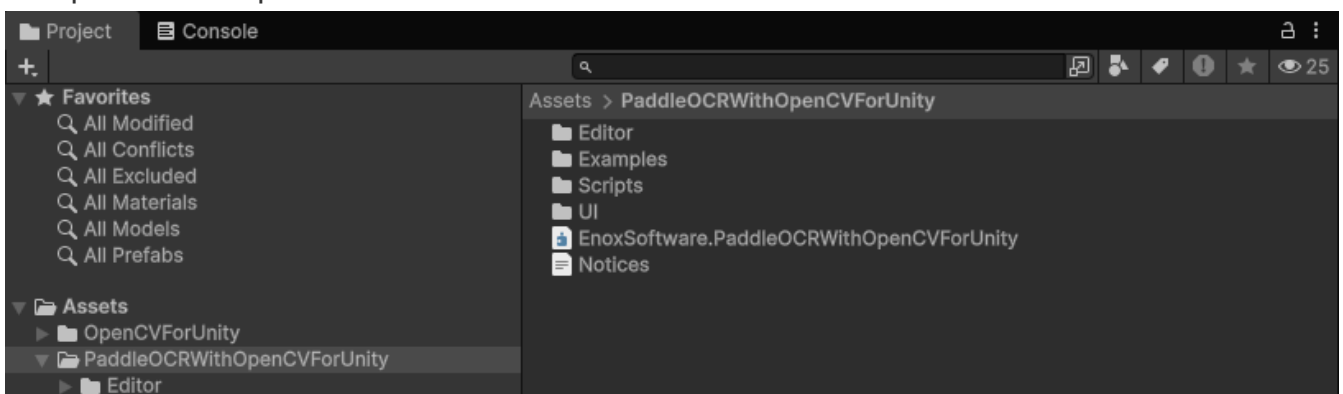
• WebGL

- Enable **SIMD** / multi-threading in **OpenCV for Unity** **Setup Tools** when aiming for better performance.

 The browser you run must support multi-threading and SIMD.



6. Setup is now complete.





For each Example, please see [Example Scenes Description](#).

Example Scenes Description

The Example Scenes in **PaddleOCR With OpenCV for Unity** are structured to let you learn the OCR workflow progressively.

Main Menu

- **PaddleOCRWithOpenCVForUnityExample**: Example scene launcher (version info, links to all examples, **ShowSystemInfo**, **ShowLicense**).

Examples Folder

- **TutorialExample**: Minimal example—attach `PaddleOCRComponent`, call `Submit(Texture2D)`, receive `PaddleOCRParsedResult` via `UnityEvent`.
- **BenchmarkExample**: Runs OCR on an assigned `Texture2D`, draws detection boxes and recognized text on a `RawImage`, and reports inference timing.
- **MultiSourceToMatHelperExample**: Real-time OCR from **WebCam**, video files, images, and other sources using `MultiSourceToMatHelper`; supports **Sentis** backend selection and sync/async modes.
- **HandwritingOCRExample**: Draw on a handwriting canvas; OCR runs periodically while drawing and once after stroke completion.
- **ShowLicense**: Displays license information for this asset and bundled third-party components.

PaddleOCR Model Files

By default, `PaddleOCRComponent` loads models from

Assets/StreamingAssets/PaddleOCRWithOpenCVForUnity/ (StreamingAssetsRootFolder).

Typical file layout (names may vary by language pack):

Role	Example file
Detection	PP-OCrv5_mobile_det_infer_640x640.sentis PP-OCrv5_mobile_det_infer_640x640.onnx
Classification	PP-LCNet_x0_25_textline_ori_infer.sentis PP-LCNet_x0_25_textline_ori_infer.onnx
Recognition	multi/PP-OCrv5_mobile_rec_infer.sentis multi/PP-OCrv5_mobile_rec_infer.onnx
Dictionary	multi/PP-OCrv5_rec_character_dict.txt

Set `PaddleOCRComponent.RecLanguage` in the **Inspector** to switch recognition language folders (**multi**, **en**, **latin**, etc.).

Supported Recognition Languages

The following recognition models are supported under

StreamingAssets/PaddleOCRWithOpenCVForUnity/. Language coverage follows [PP-OCrv5 Multilingual Text Recognition \(PaddleOCR\)](#).

Folder name	Model name	Supported languages
multi	PP-OCrv5_mobile_rec	Simplified Chinese, Pinyin, Traditional Chinese, English, Japanese
korean	korean_PP-OCrv5_mobile_rec	Korean, English
latin	latin_PP-OCrv5_mobile_rec	French, German, Afrikaans, Italian, Spanish, Bosnian, Portuguese, Czech, Welsh, Danish, Estonian, Irish, Croatian, Uzbek, Hungarian, Serbian (Latin), Indonesian, Occitan, Icelandic, Lithuanian, Maori, Malay, Dutch, Norwegian, Polish, Slovak, Slovenian, Albanian, Swedish, Swahili, Tagalog, Turkish, Latin, Azerbaijani, Kurdish, Latvian, Maltese, Pali, Romanian,

Folder name	Model name	Supported languages
		Vietnamese, Finnish, Basque, Galician, Luxembourgish, Romansh, Catalan, Quechua
eslav	eslav_PP-OCrv5_mobile_rec	Russian, Belarusian, Ukrainian, English
th	th_PP-OCrv5_mobile_rec	Thai, English
el	el_PP-OCrv5_mobile_rec	Greek, English
en	en_PP-OCrv5_mobile_rec	English (<i>optimized for English scenarios</i>)
cyrillic	cyrillic_PP-OCrv5_mobile_rec	Russian, Belarusian, Ukrainian, Serbian (Cyrillic), Bulgarian, Mongolian, Abkhazian, Adyghe, Kabardian, Avar, Dargin, Ingush, Chechen, Lak, Lezgin, Tabasaran, Kazakh, Kyrgyz, Tajik, Macedonian, Tatar, Chuvash, Bashkir, Malian, Moldovan, Udmurt, Komi, Ossetian, Buryat, Kalmyk, Tuvan, Sakha, Karakalpak, English
arabic	arabic_PP-OCrv5_mobile_rec	Arabic, Persian, Uyghur, Urdu, Pashto, Kurdish, Sindhi, Balochi, English
devanagari	devanagari_PP-OCrv5_mobile_rec	Hindi, Marathi, Nepali, Bihari, Maithili, Angika, Bhojpuri, Magahi, Santali, Newari, Konkani, Sanskrit, Haryanvi, English
te	te_PP-OCrv5_mobile_rec	Telugu, English
ta	ta_PP-OCrv5_mobile_rec	Tamil, English

Place each model's `.sentis` file and character dictionary under the corresponding folder (e.g. `StreamingAssets/PaddleOCRWithOpenCVForUnity/latin/`).

You can reduce build size by deleting recognition language folders under `StreamingAssets/PaddleOCRWithOpenCVForUnity/` that your app does not use (for example, remove `latin/` and `arabic/` if you only need `multi/`). Keep the folder that matches `PaddleOCRComponent.RecLanguage`, and do not remove shared detection or classification model

PaddleOCR Model Conversion (pdiparams → onnx)

1. Obtain the official **PP-OCRv5** model files (`.pdiparams` , `.pdmodel` , `.json`) for detection (**det**), classification (**cls**), and recognition (**rec**).

Official models can be downloaded from the [OCR Pipeline Usage Tutorial \(PaddleOCR\)](#).

2. Use the official PaddleOCR export tools to convert each model to `.onnx` :

- Detection (`PP-OCRv5_mobile_det_infer`):

```
paddle2onnx --model_dir ./PP-OCRv5_mobile_det_infer --model_filename  
inference.json --params_filename inference.pdiparams --save_file ./PP-  
OCRv5_mobile_det_infer_tmp.onnx --opset_version 10 --enable_onnx_checker  
True
```

```
python -m paddle2onnx.optimize --input_model ./PP-  
OCRv5_mobile_det_infer_tmp.onnx --output_model ./PP-  
OCRv5_mobile_det_infer.onnx --input_shape_dict '{"x":[1,3,640,640]}'"
```

- Classification (`PP-LCNet_x0_25_textline_ori_infer`):

```
paddle2onnx --model_dir ./PP-LCNet_x0_25_textline_ori_infer --  
model_filename inference.json --params_filename inference.pdiparams --  
save_file ./PP-LCNet_x0_25_textline_ori_infer_tmp.onnx --opset_version 10 --  
enable_onnx_checker True
```

```
python -m paddle2onnx.optimize --input_model ./PP-  
LCNet_x0_25_textline_ori_infer_tmp.onnx --output_model ./PP-  
LCNet_x0_25_textline_ori_infer.onnx --input_shape_dict '{"x":[22,3,80,160]}'"
```

- Recognition (`PP-OCRv5_mobile_rec_infer`):

```
paddle2onnx --model_dir ./PP-OCRv5_mobile_rec_infer --model_filename  
inference.json --params_filename inference.pdiparams --save_file ./PP-  
OCRv5_mobile_rec_infer_tmp.onnx --opset_version 10 --enable_onnx_checker  
True
```

```
python -m paddle2onnx.optimize --input_model ./PP-  
OCRv5_mobile_rec_infer_tmp.onnx --output_model ./PP-  
OCRv5_mobile_rec_infer.onnx --input_shape_dict '{"x":[6,3,48,320]}'"
```

3. Convert the exported `.onnx` files to `.sentis` for **Unity Sentis** inference and verify they load

correctly.

? Q & A

Q1. Which **Unity** and **OpenCV for Unity** versions are required?

A1. **Unity 6000.0.83f1** or later, **OpenCV for Unity 3.0.4** or later, and `com.unity.ai.inference` (**Unity Sentis**). Enable **Unity Sentis** integration in **OpenCV for Unity Setup Tools** before using this asset.

Q2. Models are not loaded; console shows detection model errors.

A2. Confirm `.sentis` model files and dictionaries are under `Assets/StreamingAssets/PaddleOCRWithOpenCVForUnity/` and that paths in `PaddleOCRComponent` match your folder layout. If models are still under `Assets/PaddleOCRWithOpenCVForUnity/StreamingAssets/`, open `Tools > PaddleOCR With OpenCV For Unity > Open Setup Tools` and click `Move StreamingAssets Folder`. Also confirm `com.unity.ai.inference` is installed via **OpenCV for Unity Setup Tools**. Check `PaddleOCRComponent` log output for the resolved `StreamingAssets` paths.

Q3. Level 'TutorialExample' (-1) could not be loaded appears.

A3. Open `Tools > PaddleOCR With OpenCV For Unity > Open Setup Tools` and click `Add Example Scenes In Build`, or add all `***.unity` files under `Assets/PaddleOCRWithOpenCVForUnity/Examples/` to `Build Settings` → `Scenes In Build`.

Q4. Is **Windows10 UWP** supported?

A4. No. **Windows10 UWP** is not supported. Scripts are wrapped with `#if !UNITY_WSA_10_0` and require **.NET Standard 2.1** (same as **OpenCV for Unity** dnn-related examples on supported platforms).

Q5. How do I switch recognition language?

A5. Set `PaddleOCRComponent.RecLanguage` in the **Inspector** (or before `InitializeInferenceAsync`). Ensure the corresponding language folder and model files exist under `StreamingAssets/PaddleOCRWithOpenCVForUnity/` .

Q6. How do I use GPU inference?

A6. After `com.unity.ai.inference` is installed, place `.sentis` models under `StreamingAssets/PaddleOCRWithOpenCVForUnity/` , set `PaddleOCRComponent.InferenceBackend` to `InferenceBackendType.UnitySentis` , and set `PaddleOCRComponent.SentisBackend` in the **Inspector** (e.g. **GPUCompute**). Call `InitializeInferenceAsync` after changing.

Q7. Can I use only detection or recognition?

A7. Yes. Use `TextDetector` , `TextClassifier` , and `TextRecognizer` directly, or configure `PaddleOCRComponent` and handle partial pipeline results via `PaddleOCRPipelineUtility` and parsed result APIs.

Version Changes

1.0.2 [Common] Updated for OpenCV for Unity v3.0.4. (This asset requires OpenCVforUnity 3.0.4 or later.)

1.0.1 [Common] Improved the inference processing time. [Common] Changed the number of batches for the `TextClassifier` model.

1.0.0 [Common] Initial version.